

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 3

Part 1:

Task A — Rotation About an Arbitrary Pivot (Core)

1.

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2. We have

$$R^T = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

So when we multiply

$$R^T R = \begin{bmatrix} \cos\theta & \sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} * \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

3. We have

$$\begin{aligned} \text{Det}(R) &= \det \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * 1 \\ \det(R) &= \cos\theta * \cos\theta - (-\sin\theta) * \sin\theta = 1 \end{aligned}$$

4.

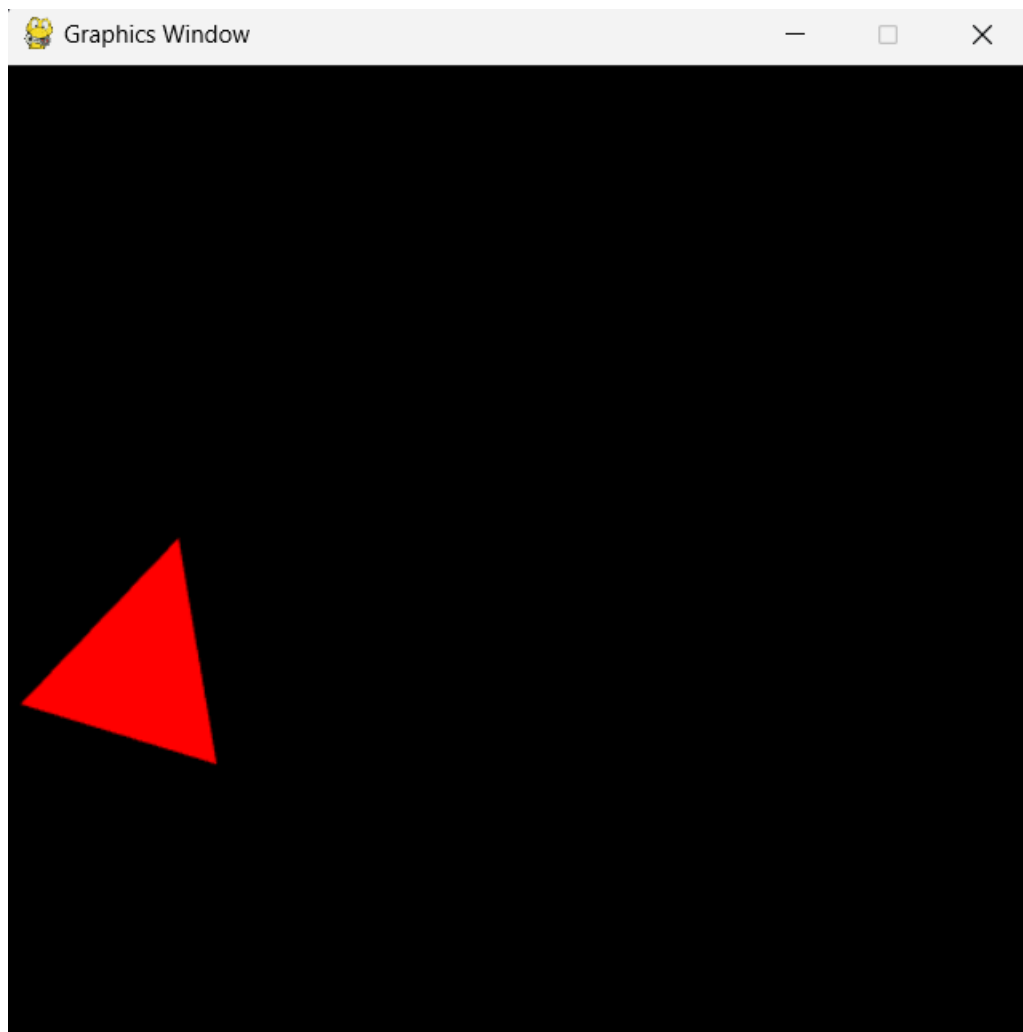
Code:

```

1 #!/usr/bin/python
2 import math
3
4 import OpenGL.GL as GL
5
6 from py3d.core.base import Base
7 from py3d.core.utils import Utils
8 from py3d.core.attribute import Attribute
9 from py3d.core.uniform import Uniform
10
11
12 class Example(Base):
13     """ Rotate a triangle moving in a circular path around the origin """
14     def initialize(self):
15         print("Initializing program...")
16         # Initialize program #
17         vs_code = """
18             in vec3 position;
19             uniform vec3 translation;
20             uniform vec3 center;
21             uniform float angle;
22             void main()
23             {
24                 float c = cos(angle);
25                 float s = sin(angle);
26                 mat2 rotation = mat2(c, -s, s, c);
27
28                 vec2 local = position.xy - center.xy;
29                 vec2 rotated = rotation * local;
30                 vec2 world = rotated + center.xy + translation.xy;
31
32                 gl_Position = vec4(world.x, world.y, position.z + translation.z, 1.0);
33             }
34         """
35         fs_code = """
36             uniform vec3 baseColor;
37             out vec4 fragColor;
38             void main()
39             {
40                 fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
41             }
42         """
43         self.program_ref = Utils.initialize_program(vs_code, fs_code)
44         # Render settings (optional) #
45         # Specify color used when clearing
46         GL.glClearColor(0.0, 0.0, 0.0, 1.0)
47         # Set up vertex array object #
48         vao_ref = GL.glGenVertexArrays(1)
49         GL.glBindVertexArray(vao_ref)
50         # Set up vertex attribute #
51         self.position_data = [[ 0.0, 0.2, 0.0],
52                               [ 0.2, -0.2, 0.0],
53                               [-0.2, -0.2, 0.0]]
54         self.vertex_count = len(self.position_data)
55         position_attribute = Attribute('vec3', self.position_data)
56         position_attribute.associate_variable(self.program_ref, 'position')
57
58         cx = sum(v[0] for v in self.position_data) / self.vertex_count
59         cy = sum(v[1] for v in self.position_data) / self.vertex_count
60         cz = 0.0
61         # Set up uniforms #
62         self.translation = Uniform('vec3', [0.0, 0.0, 0.0])
63         self.translation.locate_variable(self.program_ref, 'translation')
64
65         self.center = Uniform('vec3', [cx, cy, cz])
66         self.center.locate_variable(self.program_ref, 'center')
67
68         self.angle = Uniform('float', 0.0)
69         self.angle.locate_variable(self.program_ref, 'angle')
70
71         self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
72         self.base_color.locate_variable(self.program_ref, 'baseColor')
73
74         self.orbit_radius = 0.75
75         self.orbit_speed = 1.0 # radians per second
76         self.spin_speed = 2.0
77         self.center_speed = 0.5
78
79     def update(self):
80         """ Update data """
81         t = self.time
82         self.translation.data[0] = 0.75 * math.cos(self.time)
83         self.translation.data[1] = 0.75 * math.sin(self.time)
84         self.angle.data = self.spin_speed * t
85
86         # update pivot position
87         distance = self.center_speed * self.delta_time
88         if self.input.is_key_pressed('j'):
89             self.center.data[0] -= distance
90         if self.input.is_key_pressed('i'):
91             self.center.data[0] += distance
92         if self.input.is_key_pressed('l'):
93             self.center.data[1] += distance
94         if self.input.is_key_pressed('k'):
95             self.center.data[1] -= distance
96
97         if self.input.is_key_pressed('h'):
98             self.spin_speed += 0.5 * self.delta_time
99         if self.input.is_key_pressed('n'):
100             self.spin_speed = max(0.5, self.spin_speed - 0.5 * self.delta_time)
101         # Render color buffer with specified color
102         GL.glClear(GL.GL_COLOR_BUFFER_BIT)
103         GL.glUseProgram(self.program_ref)
104         self.translation.upload_data()
105         self.center.upload_data()
106         self.angle.upload_data()
107         self.base_color.upload_data()
108         GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
109
110
111 # Instantiate this class and run the program
112 Example().run()

```

Result:



Task B — Order Matters: $T \cdot R$ vs $R \cdot T$ (Core)

```
# Left triangle (global)
self.angle.data = theta
self.translation.data = [-0.5 + self.radius * math.cos(t),
                        0.0 + self.radius * math.sin(t)]
self.angle.upload_data()
self.translation.upload_data()
self.base_color.data = [1.0, 0.0, 0.0]
self.base_color.upload_data()
GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
```

```
# Right triangle (local)
self.angle.data = theta
self.translation.data = [0.5 + self.radius * math.cos(t),
                        0.0 + self.radius * math.sin(t)]
self.angle.upload_data()
self.translation.upload_data()
self.base_color.data = [0.0, 0.5, 1.0]
self.base_color.upload_data()
GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)
```

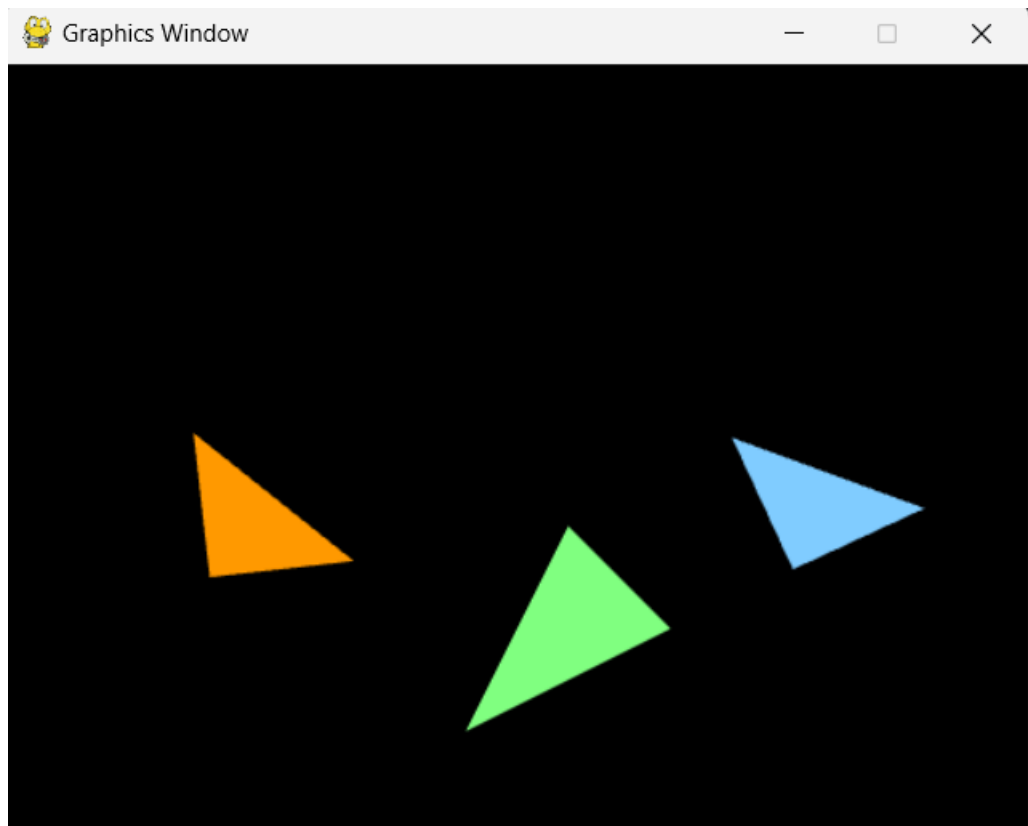
Global Motion:

- The triangle rotates around the origin (global axis), meaning its entire position moves along a circular path around the world origin.
- This is called global motion because the rotation occurs after translation, so the pivot is at the origin, not at the triangle itself.

Local Motion:

- The triangle rotates around its own center while moving along a path defined by the translation.
- This is called local motion because the rotation happens in the triangle's own local coordinate system, pivoting at its center.

Task C — Advanced Tracks (Choose 1 for bonus)



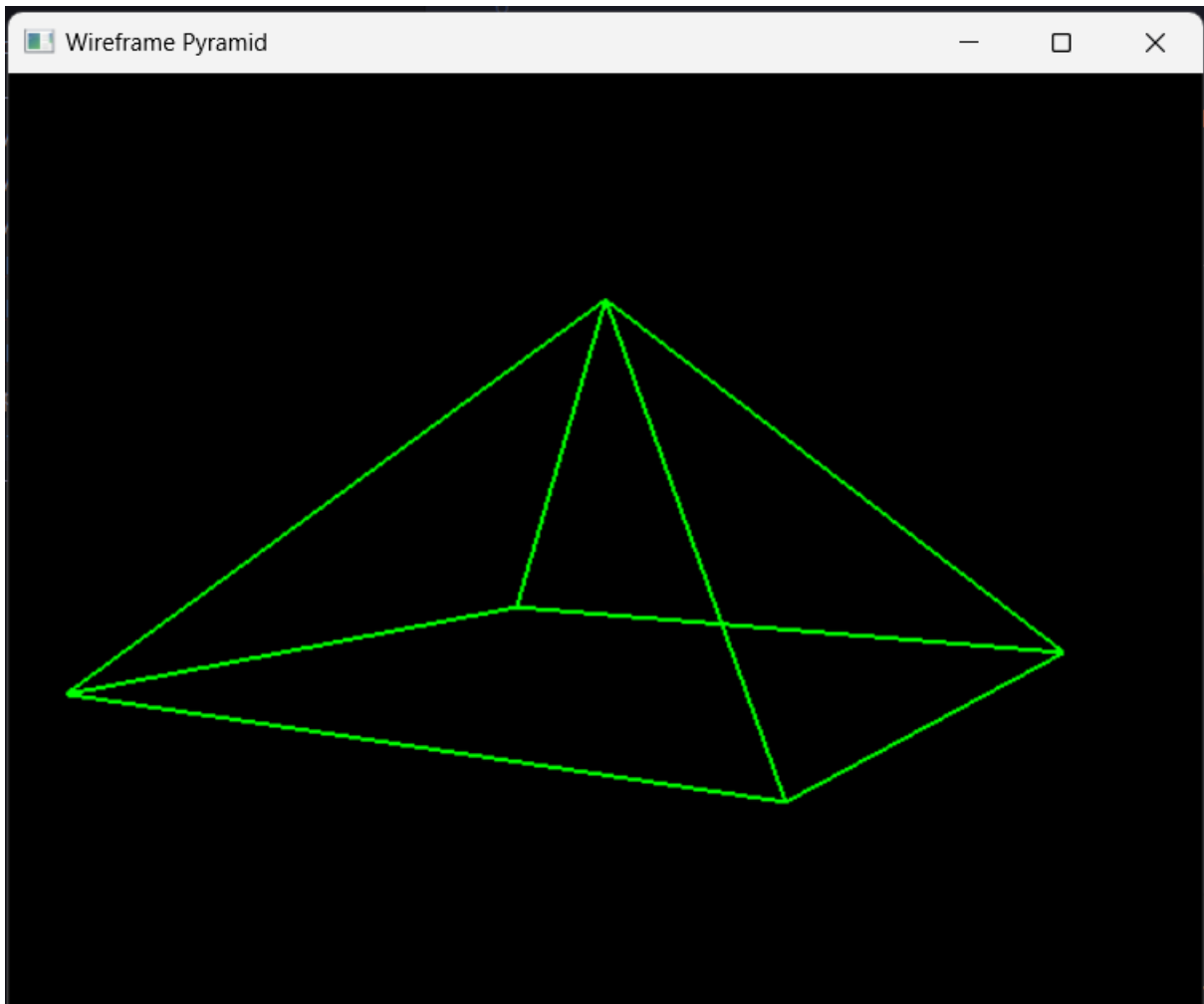
Part 2:

Task A — Draw a wireframe square pyramid

```

1  from OpenGL.GL import *
2  from OpenGL.GLUT import *
3  from OpenGL.GLU import *
4
5  angle = 0
6
7  def draw_pyramid():
8      global angle
9      glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
10     glLoadIdentity()
11
12     glTranslatef(0, 0, -8)
13     glRotatef(10, 1, 0, 0)
14     glRotatef(angle, 0, 1, 0)
15
16     glColor3f(0, 1, 0)
17     glLineWidth(2)
18
19     h = 2.0
20     s = 2.0
21     apex = [0.0, h, 0.0]
22     v0 = [-s, 0.0, -s]
23     v1 = [ s, 0.0, -s]
24     v2 = [ s, 0.0,  s]
25     v3 = [-s, 0.0,  s]
26
27     glBegin(GL_LINES)
28     # Base edges
29     glVertex3fv(v0); glVertex3fv(v1)
30     glVertex3fv(v1); glVertex3fv(v2)
31     glVertex3fv(v2); glVertex3fv(v3)
32     glVertex3fv(v3); glVertex3fv(v0)
33     # Sides
34     for v in [v0, v1, v2, v3]:
35         glVertex3fv(apex); glVertex3fv(v)
36     glEnd()
37
38     glutSwapBuffers()
39 def update(value):
40     global angle
41     angle += 0.5
42     glutPostRedisplay()
43     glutTimerFunc(16, update, 0)
44
45 def main():
46     glutInit()
47     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)
48     glutInitWindowSize(600, 600)
49     glutCreateWindow(b"Wireframe Pyramid")
50     glEnable(GL_DEPTH_TEST)
51     glClearColor(0, 0, 0, 1)
52
53     glMatrixMode(GL_PROJECTION)
54     glLoadIdentity()
55     gluPerspective(45, 1.0, 0.1, 50.0)
56     glMatrixMode(GL_MODELVIEW)
57
58     glutDisplayFunc(draw_pyramid)
59     glutTimerFunc(0, update, 0)
60     glutMainLoop()
61
62 if __name__ == "__main__":
63     main()
64

```



Task B — Theory: design the matrices

1. Homogeneous translation

$$T(tx, ty, tz) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ tx & ty & tz & 1 \end{bmatrix}$$

Homogeneous coordinates are required because translation is not a linear transform in 3×3 . By adding $w = 1$, translation becomes an affine transform.

2. Scale

$$S(s) = \begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Normals become distorted, breaking lighting calculations. We must transform normals by the inverse-transpose of the model matrix to preserve correct direction and length.

3. Axis rotations

a)

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

b)

- **Global rotation:** $R * M \rightarrow$ rotate the model about world axes.
- **Local rotation:** $M * R \rightarrow$ rotate the model about its own local axes.

4. Shear

$$Sh_{xy} = \begin{bmatrix} 1 & k_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5. Perspective

$$P = \begin{bmatrix} \frac{1}{\tan(fovy/2) \cdot aspect} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(fovy/2)} & 0 & 0 \\ 0 & 0 & -\frac{z_{far} + z_{near}}{z_{far} - z_{near}} & -1 \\ 0 & 0 & -\frac{2z_{far}z_{near}}{z_{far} - z_{near}} & 0 \end{bmatrix}$$

After the vertex shader:

- Vertex is transformed to clip-space: $v_{clip} = P * M * v_{model}$
- The w component in clip-space is used for perspective divide:

$$x_{ndc} = x / w,$$

$$y_{ndc} = y / w,$$

$$z_{ndc} = z / w,$$

- This converts coordinates to Normalized Device Coordinates (NDC) before rasterization.

Task C — Implementation: interactive transforms

```

1 from OpenGL.GL import *
2 from OpenGL.GLUT import *
3 from OpenGL.GLU import *
4 import numpy as np
5
6 angle = 0
7 M = np.identity(4, dtype=np.float32)
8
9 def translate(tx, ty, tz):
10     T = np.identity(4, dtype=np.float32)
11     T[3, :3] = [tx, ty, tz]
12     return T
13
14 def rotate_z(theta_deg):
15     theta = np.radians(theta_deg)
16     R = np.identity(4, dtype=np.float32)
17     c, s = np.cos(theta), np.sin(theta)
18     R[0,0], R[0,1] = c, s
19     R[1,0], R[1,1] = -s, c
20     return R
21
22 def draw_pyramid():
23     global angle, M
24     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
25     glLoadIdentity()
26
27     glTranslatef(0, 0, -8)
28     glRotatef(10, 1, 0, 0)
29     glRotatef(angle, 0, 1, 0)
30
31     glMultMatrixf(M.T)
32
33     glColor3f(0, 1, 0)
34     glLineWidth(2)
35
36     h = 2.0
37     s = 2.0
38     apex = [0.0, h, 0.0]
39     v0 = [-s, 0.0, -s]
40     v1 = [ s, 0.0, -s]
41     v2 = [ s, 0.0,  s]
42     v3 = [-s, 0.0,  s]
43
44     glBegin(GL_LINES)
45     glVertex3fv(v0); glVertex3fv(v1)
46     glVertex3fv(v1); glVertex3fv(v2)
47     glVertex3fv(v2); glVertex3fv(v3)
48     glVertex3fv(v3); glVertex3fv(v0)
49     for v in [v0, v1, v2, v3]:
50         glVertex3fv(apex); glVertex3fv(v)
51     glEnd()
52
53     glutSwapBuffers()
54
55 def keyboard(key, x, y):
56     global M
57     key = key.decode('utf-8').lower()
58     delta = 0.2
59     rot = 5
60
61     if key == 'w':
62         M = translate(0, delta, 0) @ M
63     elif key == 's':
64         M = translate(0, -delta, 0) @ M
65     elif key == 'd':
66         M = translate(delta, 0, 0) @ M
67     elif key == 'a':
68         M = translate(-delta, 0, 0) @ M
69     elif key == 'z':
70         M = translate(0, 0, delta) @ M
71     elif key == 'x':
72         M = translate(0, 0, -delta) @ M
73     elif key == 'q':
74         M = rotate_z(rot) @ M
75     elif key == 'e':
76         M = rotate_z(-rot) @ M
77
78     elif key == 'i':
79         M = M @ translate(0, delta, 0)
80     elif key == 'k':
81         M = M @ translate(0, -delta, 0)
82     elif key == 'j':
83         M = M @ translate(-delta, 0, 0)
84     elif key == 'l':
85         M = M @ translate(delta, 0, 0)
86     elif key == 'u':
87         M = M @ rotate_z(rot)
88     elif key == 'o':
89         M = M @ rotate_z(-rot)
90
91     glutPostRedisplay()
92
93 def update(value):
94     global angle
95     angle += 0.5
96     glutPostRedisplay()
97     glutTimerFunc(16, update, 0)
98
99 def main():
100     glutInit()
101     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)
102     glutInitWindowSize(600, 600)
103     glutCreateWindow(b"Wireframe Pyramid")
104     glEnable(GL_DEPTH_TEST)
105     glClearColor(0,0,0,1)
106
107     glMatrixMode(GL_PROJECTION)
108     glLoadIdentity()
109     gluPerspective(45,1.0,0.1,50.0)
110     glMatrixMode(GL_MODELVIEW)
111
112     glutDisplayFunc(draw_pyramid)
113     glutKeyboardFunc(keyboard)
114     glutTimerFunc(0, update, 0)
115     glutMainLoop()
116
117 if __name__ == "__main__":
118     main()
119

```

Part 3:

Theory

1. Pyramid cross-section

The allowed half-extent at height y is:

$$h(y) = s \left(1 - \frac{y}{a_y} \right), \quad 0 \leq y \leq a_y$$

2. Reflection rule

- When a particle crosses a plane at $x = \pm h$
 - **Position correction:** clamp to plane: $x = \pm h$
 - **Velocity update:** flip only the **normal component** $v_x \rightarrow -v_x$
- Why not flip both position and velocity naïvely:
 - Finite floating-point precision can push the particle slightly beyond the boundary.
 - Flipping both can trap the particle oscillating inside/outside the plane, causing jitter or “sticking.”

3. Space of simulation

- Simulate in pyramid's local model space because:
 - Particle positions are relative to the object.
 - Rotations or translations of the pyramid don't require updating all particle positions globally.
- If simulated in world space while object rotates:
 - The pyramid rotates, but particles don't rotate with it $\rightarrow$ they appear to “slide off” or break containment.

4. Shared and per-particle transforms
 - Shared modelMatrix: applies to all particles relative to the pyramid.
 - Per-particle attributes needed in some cases:
 1. Billboarding: particle always faces the camera (rotation varies per particle).
 2. Per-particle scale/spin: individual particle size or rotation animation differs from others.

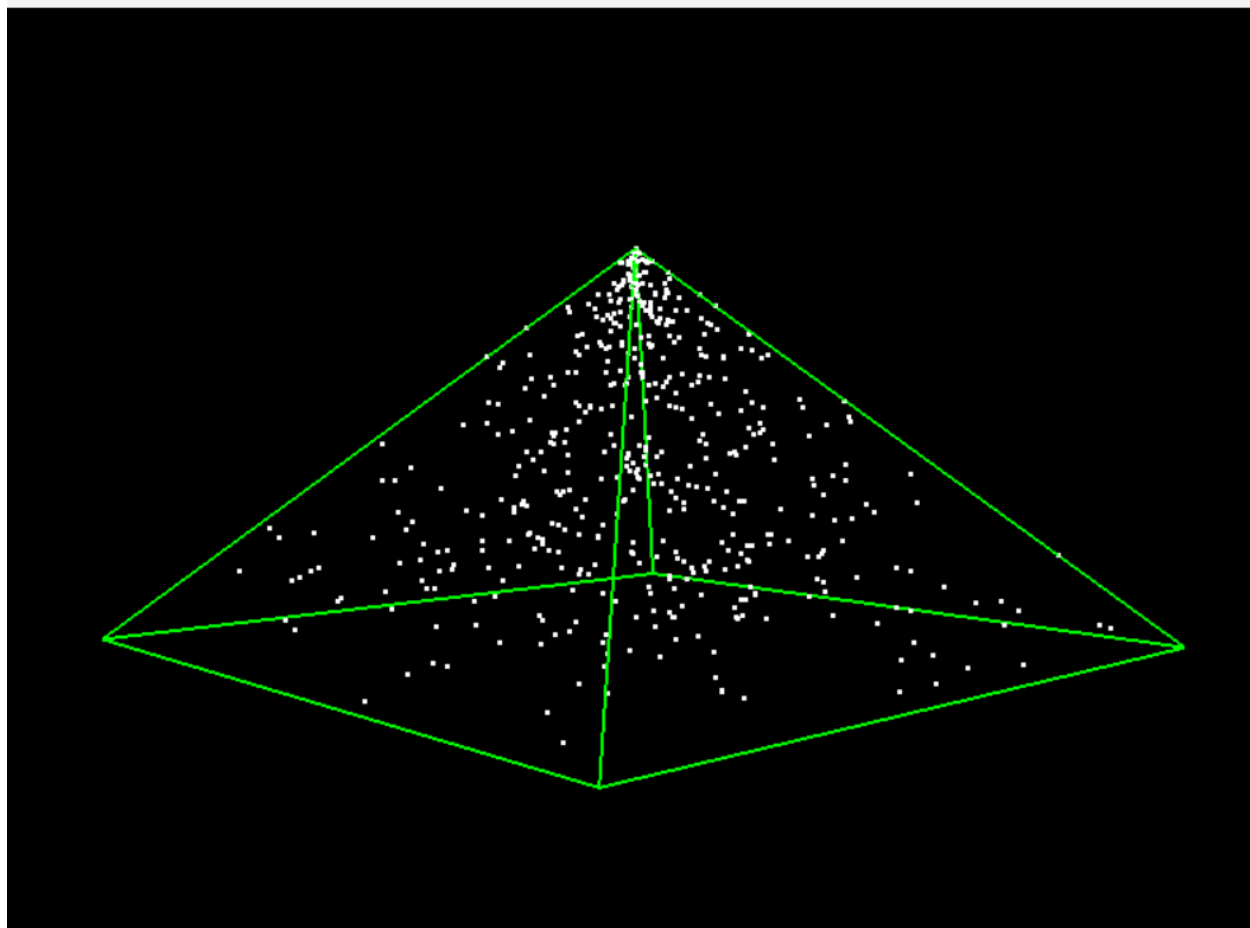
Task

A.

```

1 from OpenGL.GL import *
2 from OpenGL.GLUT import *
3 from OpenGL.GLU import *
4 import numpy as np
5
6 # --- Pyramind and particles ---
7 h = 2.0 # base height
8 w = 2.0 # base width
9 n = 100 # number of particles
10 alpha = 0.009 # damping factor
11 dt = 1/60.0 # time step
12
13 # --- Positions and velocities ---
14 positions = np.zeros((n,3), dtype=np.float32)
15 velocities = np.zeros((n,3), dtype=np.float32)
16
17 # --- Create particles inside the pyramid ---
18 for i in range(n):
19     x = np.random.uniform(0, w)
20     y = x * (1 - h/w)
21     z = np.random.uniform(0, h)
22     x = np.random.uniform(-h, h)
23     positions[i] = [x, y, z]
24     velocities[i] = np.random.uniform(-0.5, 0.5, 3)
25
26 # --- Model matrix for pyramind's particle plane ---
27 M = np.identity(4, dtype=np.float32)
28
29 # --- Particle moving ---
30 def move_particles():
31     # given positions, velocities
32     for i in range(n):
33         positions[i] += velocities[i] * dt
34
35         # Bounce off
36         if positions[i,1] < 0:
37             positions[i,1] = 0
38             velocities[i,1] *= -1
39         elif positions[i,1] > w:
40             positions[i,1] = w
41             velocities[i,1] *= -1
42
43         # Bounce current, hit floor
44         h_current = x * (1 - positions[i,1]/w)
45
46         # Bounce on z walls
47         if positions[i,0] < -h_current:
48             positions[i,0] = -h_current
49             velocities[i,0] *= -1
50         elif positions[i,0] > h_current:
51             positions[i,0] = h_current
52             velocities[i,0] *= -1
53
54         # Bounce on x walls
55         if positions[i,2] < -h_current:
56             positions[i,2] = -h_current
57             velocities[i,2] *= -1
58         elif positions[i,2] > h_current:
59             positions[i,2] = h_current
60             velocities[i,2] *= -1
61
62         # Damping damping
63         velocities[i] *= alpha
64
65 # --- Draw the pyramind ---
66 def draw_pyramid():
67     glClear(GL_COLOR_BUFFER_BIT)
68     glLoadIdentity()
69     h = 2
70     w = 2
71     base = [w, w, 0]
72     v1 = [w, 0, 0]
73     v2 = [-w, 0, 0]
74     v3 = [-w, w, 0]
75
76     glBegin(GL_TRIANGLES)
77     # Base
78     glVertex3f(w,0,0); glVertex3f(v1);
79     glVertex3f(v1); glVertex3f(v2);
80     glVertex3f(v2); glVertex3f(v3);
81     glVertex3f(v3); glVertex3f(w,0,0);
82     # sides
83     for v in [v1,v2,v3]:
84         glVertex3f(w,0,0); glVertex3f(v);
85     glEnd()
86
87 # --- Particle moving ---
88 def move_particles():
89     glClear(GL_COLOR_BUFFER_BIT)
90     glLoadIdentity()
91     # translate matrix
92     for p in positions:
93         glVertex3f(p)
94     glEnd()
95
96 # --- Time ---
97 def timer(value):
98     # draw angle
99     angle = 0.5
100     rotate(angle)
101     move_particles()
102     glutPostRedisplay()
103     glutTimerFunc(100, timer, 0)
104
105 # --- Keyboard (translation only) ---
106 def translate(x,y,z):
107     # translate matrix
108     T = np.identity(4, dtype=np.float32)
109     T[3,1] = [x,y,z]
110     return T
111
112 def keyboard(key, x, y):
113     # given M
114     key = key.lower('a-z').lower()
115     delta = 0.2
116
117     # Rotate, translate
118     if key == 'w': M = translate(0,delta,0) @ M
119     elif key == 's': M = translate(0,-delta,0) @ M
120     elif key == 'd': M = translate(delta,0,0) @ M
121     elif key == 'f': M = translate(-delta,0,0) @ M
122     elif key == 'l': M = translate(0,delta,0) @ M
123     elif key == 'r': M = translate(0,0,delta) @ M
124
125     # Rotate, translate
126     elif key == '1': M = M @ translate(0,delta,0)
127     elif key == '2': M = M @ translate(0,-delta,0)
128     elif key == '3': M = M @ translate(delta,0,0)
129     elif key == '4': M = M @ translate(-delta,0,0)
130     elif key == '5': M = M @ translate(0,0,delta)
131     elif key == '6': M = M @ translate(0,0,-delta)
132
133     # Rotate
134     glPushMatrix()
135     # rotate
136     for mat in [M]:
137         glutInit()
138         glutInitDisplayMode(GLUT_DEPTH | GLUT_RGBA | GLUT_DOUBLE)
139         glutInitWindowPosition(100,100)
140         glutCreateWindow("Pyramind Particle Simulation - Part 1")
141         glClearColor(0,0,0,0)
142         glClear(GL_COLOR_BUFFER_BIT)
143
144         # Matrix
145         glMatrixMode(GL_PROJECTION)
146         glLoadIdentity()
147         gluPerspective(45,1,0.1,100.0)
148         glMatrixMode(GL_MODELVIEW)
149
150         glutDisplayFunc(draw_pyramid)
151         glutKeyboardFunc(keyboard)
152         glutTimerFunc(0, timer, 0)
153         glutMainLoop()
154
155 if __name__ == '__main__':
156     main()

```



B.

```

from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *
import numpy as np

# --- Physical and particle vars ---
h = 2.0 # base height
w = 2.0 # base width
n = 100 # number of particles
alpha = 0.999 # damping factor
dt = 1/60.0 # time step

# --- Simulation parameters ---
positions = np.zeros((n,3), dtype=np.float32)
velocities = np.zeros((n,3), dtype=np.float32)

# --- Create particles from the ground ---
for i in range(n):
    x = np.random.uniform(0, w)
    y = h * (1 + 0.5*np.random.random())
    z = np.random.uniform(0, h)
    x = np.random.uniform(0, h)
    positions[i] = [x, y, z]
    velocities[i] = np.random.uniform(-0.5, 0.5, 3)

# --- Model matrix for pyramide's particle plane ---
M = np.identity(4, dtype=np.float32)

# --- Particle moving ---
def move_particles():
    global positions, velocities
    for i in range(n):
        positions[i] += velocities[i] * dt

        # Bounce on x walls
        if positions[i,0] < 0:
            positions[i,0] = 0
            velocities[i,0] *= -1
        elif positions[i,0] > w:
            positions[i,0] = w
            velocities[i,0] *= -1

        # Bounce on z walls
        if positions[i,2] < -h_current:
            positions[i,2] = -h_current
            velocities[i,2] *= -1
        elif positions[i,2] > h_current:
            positions[i,2] = h_current
            velocities[i,2] *= -1

        velocities[i] *= alpha

# --- Pyramide drawing ---
def draw_pyramid():
    glClear(GL_COLOR_BUFFER_BIT)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluLookAt(0, 0, 0, 0, 0, 0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    for v in [w,vz,vj]:
        glVertex3f(v,0,0); glVertex3f(v,h,0)
    glEnd()

# --- Particle moving ---
def draw_particles():
    glClear(GL_COLOR_BUFFER_BIT)
    glPointSize(100)
    glBegin(GL_POINTS)
    for p in positions:
        glVertex3f(p)
    glEnd()

# --- Main loop ---
angle = 0.0
def draw():
    global angle
    glClear(GL_COLOR_BUFFER_BIT)
    glLoadIdentity()
    gluLookAt(0,0,0,0,0,0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    draw_pyramid()
    draw_particles()
    glFlush()

def timer(value):
    global angle
    angle += 0.01
    move_particles()
    draw()
    glutTimerFunc(1000, timer, 0)

# --- Keyboard (translation keys) ---
def translate(tx,ty,tz):
    T = np.identity(4, dtype=np.float32)
    T[1,3] = [tx,ty,tz]
    return T

def keyboard(key,x,y):
    global M
    key = key.encode('utf-8').lower()
    dz = 0.2

    # Down (move up)
    if key == 'w': M = translate(0,delta,0) @ M
    elif key == 's': M = translate(0,-delta,0) @ M
    elif key == 'a': M = translate(-delta,0,0) @ M
    elif key == 'd': M = translate(delta,0,0) @ M
    elif key == 'z': M = translate(0,0,delta) @ M
    elif key == 'x': M = translate(0,0,-delta) @ M

    # Left (rot matrix)
    elif key == 'q': M = rotate(delta,0,0) @ M
    elif key == 'e': M = rotate(-delta,0,0) @ M
    elif key == 'r': M = rotate(delta,0,0) @ M
    elif key == 'f': M = rotate(-delta,0,0) @ M
    elif key == 'v': M = rotate(0,delta,0) @ M
    elif key == 'b': M = rotate(0,-delta,0) @ M

    glOrtho(0,1,0,1,0,1)

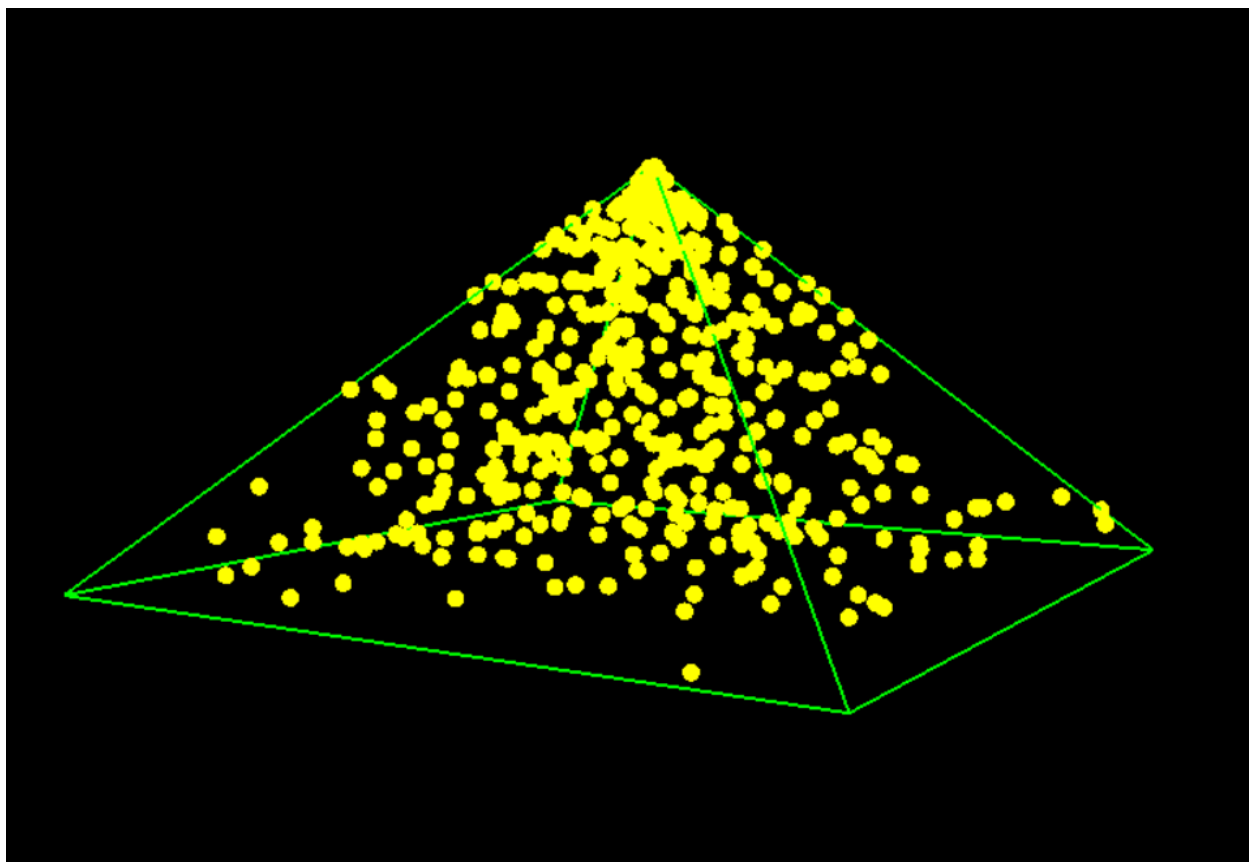
def main():
    glutInit()
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH)
    glutInitWindowSize(800,800)
    glutCreateWindow("Particle Simulation - Part 1")
    glEnable(GL_DEPTH_TEST)
    glEnable(GL_BLEND)
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
    glClearColor(0.0,0.0,0.0,1.0)

    glMatrixMode(GL_PROJECTION)
    gluLookAt(0,0,0,0,0,0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

    glutDisplayFunc(draw)
    glutKeyboardFunc(keyboard)
    glutTimerFunc(0, timer, 0)
    glutMainLoop()

if __name__ == '__main__':
    main()

```



C.

[illegible]

